University of Hertfordshire UH

School of Physics, Engineering and Computer Science

Advanced Computer Science Masters Project

Final Project Report

Project Title:

SecureNet-IDS: A Machine Learning–Based Framework for Network Anomaly Detection Using the UNSW-NB15 Dataset

By

Muhammad Naveed Ashfaq

Student ID

24065592

Supervisor: *Dr. Lora Tonga*

STATEMENT OF DECLARATION

This report is presented to partially meet the requirements for the Master of Science degree in Computer Networks and Systems Security at the University of Hertfordshire. I recognize the University's regulations on academic misconduct and am aware that infringements may lead to penalties, including failure of the project module or the entire program. I affirm that I have read and understood the applicable policies covering academic integrity, plagiarism, and assessment offences (UPR AS/C/6.1, Part 7, and UPR AS/C/5, Section 3.6).

I certify that this submission is entirely my own work and that all sources, whether published or unpublished, have been suitably acknowledged. This MSc project does not involve human participants or the administration of surveys.

I authorize this report to be made accessible on module-related platforms, so long as appropriate credit is given.

Student SRN Number: 24065592

Student Name: MUHAMMAD NAVEED ASHFAQ

Student Signature: NAVEED

DEDICATION

I dedicate this work to my loved ones, my family, friends, and colleagues, whose constant encouragement helped me carry out this research, especially through moments of doubt and difficulty.

ACKNOWLEDGMENT

I would like to thank my report supervisor, Dr. Lora Tonga, for his continuous support and guidance in the preparation of this thesis. His commitment and enthusiasm provided the important impetus for my study. Not only his academic guidance, but also his personal support was invaluable to me, and has deeply shaped my journey. I would also like to thank my family for their continued support and encouragement, without which I would not have achieved this goal.

ABSTRACT

The rapid growth of connected devices and increasing data transmission has intensified cybersecurity threats, necessitating more effective Intrusion Detection Systems (IDS). Machine learning techniques, due to their ability to handle large and complex datasets, have been widely adopted to enhance IDS performance. In this study, six machine learning classifiers—Logistic Regression, k-Nearest Neighbors (k-NN), Decision Tree, Random Forest, Gradient Boosting, and Multi-Layer Perceptron (MLP)—were evaluated using the UNSW-NB15 dataset. The results show that Random Forest achieved the best overall performance, with an accuracy of 97.70% and consistently high recall, precision, and F1-score (97.70–97.71%). Decision Tree and MLP also demonstrated strong performance, with accuracies of 96.68% and 96.67%, respectively, providing a good balance between effectiveness and efficiency. Gradient Boosting achieved moderate performance (95.73%), while k-NN (94.84%) and Logistic Regression (92.56%) showed comparatively lower accuracy. The findings indicate that ensemble-based methods, particularly Random Forest, are highly effective for network intrusion detection. However, model selection should consider not only predictive performance but also computational efficiency and suitability for real-time applications.

Keywords: Machine Learning, Intrusion Detection, UNSW-NB15, Anomaly Detection, XGBoost, Performance Evaluation.

CONTENTS

1. **CHAPTER I: INTRODUCTION** 8
1.1. Introduction 8
1.2. Problem Statement 9
1.3. Motivation for This Study 9
1.4. Research Statement 9
1.5. Research Questions 9
1.6. Goals and Objectives 10
2. **CHAPTER II: LITERATURE REVIEW** 11
2.1. Literature Review 11
2.2. Summary of Literature Review 13
3. **CHAPTER III: METHODOLOGY** 15
3.1. Dataset 15
3.2. Logistic Regression Model 15
3.2.1. Mathematical Basis of Logistic Regression 16
3.3. K-Nearest Neighbors (KNN) 16
3.3.1. Mathematical Formulation of KNN 17
3.4. Decision Tree 17
3.4.1. Mathematical Basis of Decision Trees 18
3.5. Random Forest 20
3.6. Multi-Layer Perceptron (MLP) 20
3.6.1. Mathematical Formulation 21
3.7. Gradient Boosting 21
3.7.1. Mathematics of Gradient Boosting 22
3.8. ReLU (Rectified Linear Unit) 23
3.9. Adam (Adaptive Moment Estimation) 23
3.10. Performance Metrics 23
3.10.1. Accuracy 23
3.10.2. Precision 24

3.10.3. Recall 24
3.10.4. F1-Score 24
4. CHAPTER 4: EXPERIMENTAL OUTCOMES 25
4.1. Logistic Regression Model Assessment 25
4.1.1. Per-Class Evaluation 26
4.2. k-Nearest Neighbors Classifier 26
4.3. Decision Tree Classifier 28
4.4. Random Forest Classifier Evaluation 29
4.4.1. Performance Overview 29
4.5. Gradient Boosting Classifier Analysis 30
4.6. Multi-Layer Perceptron (MLP) Classifier 32
4.7. Model Comparison and Findings 32
4.8. Model Training and Prediction Time Analysis 34
4.9. Computational Cost and Latency 35
4.10. Configuration of the Machine Learning Models 36
4.11. Machine Learning Bolsters Anomaly Detection 36
5. CONCLUSIONS AND FUTURE DIRECTIONS 38
5.1. Conclusions 38
5.2. Future Research 38
BIBLIOGRAPHY 40

1. CHAPTER I: INTRODUCTION

1.1. Introduction

Over the past 20 years, the Internet has become an integral part of our everyday lives and it is predicted that more than 50 billion devices will be connected by 2020 Ouafiq et al., 2022. With the growing interdependence of digital ecosystems, network security and cyber threat mitigation has become more complex. Critical sectors like financial services, healthcare systems and IoT devices are now more vulnerable to potential intrusions. Such security breaches lead to losses of billions of dollars every year and significantly deter the reliability of the system. The growing importance of cybersecurity, particularly in the context of intrusion detection, has been well emphasized Abiodun et al., 2021. A significant challenge in many systems today is that security-related data specification is often overlooked in the design process. This can affect the performance of machine learning models, despite the lack of extensive studies of these discrepancies. While it is known these can affect the model, a more detailed analysis of the variability needs to be conducted Berthier and Sanders, 2011.In today's world, cybersecurity is a critical concern, with digital technologies continually affecting every aspect of our lives Reveron, 2012. Anomaly detection plays a crucial role in several domains - such as network security, industrial and manufacturing systems - in detecting deviations from the norm. Increasingly complex data and the burgeoning of cyber attacks have contributed to a greater need for effective anomaly detection techniques. These approaches have been considered highly adaptive and crucial in modern applications including time-series, traffic, and image systems.Growing reliance on interconnected systems has given rise to more sophisticated cyber threats, including ransomware, advanced persistent threats (APTs), zero-day exploits, and mobile-based attacks. Advances in cybersecurity research, coupled with the expansion of cloud computing and the Internet of Things (IoT), have further intensified these challenges Redhu et al., 2024. Consequently, safeguarding the integrity of digital infrastructures is more crucial than ever. According to recent figures, the global cost of cybercrime is anticipated to increase to more than $10.5 trillion annually by 2025, reflecting a significant increase in the number of attacks and their severity. Ransomware attacks happen every 10 seconds, highlighting the urgency of the threat. Additionally, the emergence of AI-powered hacking software and self-spreading malware significantly reduces the technical barriers to hacking. This has allowed both individual hackers and organised crime groups to better exploit system vulnerabilities, and at a far greater scaleMphatheni and Maluleke, 2022.

1.2. Problem Statement

In recent years, cyber threats have increased in sophistication and number due to advances in technology. These include ransomware, distributed denial-of-service attacks, and zero-day attacks. IoT devices and cloud computing may make conventional security approaches ineffective. Anomaly detection based on machine learning could provide an effective real-time response (Falowo et al., 2024). Large datasets, high false alarm rates, and large-scale implementation pose challenges to this approach. Anomaly detection is improved using UNSW-NB15. We aim to improve detection accuracy, reduce false positives, and build a scalable system. For today's cyber environment, the results should lead to a more modern intrusion detection system.

1.3. Motivation for This Study

Intrusion Detection and Prevention Systems (IDPS) are becoming more common in today's organizations to detect potential cyberattacks, vulnerabilities, and security policies (Patel et al., 2010). Consequently, with the increasing size and complexity of network infrastructures, the need to manage and classify large amounts of data has become an important research area, highlighting the need for effective anomaly detection in large-scale systems (Erhan et al., 2021). As cyberattacks grow in complexity, there is a growing need for effective security measures. In this regard, machine-learning-based anomaly detection is a critical component of cybersecurity systems (Chaabouni et al., 2019).

1.4. Research Statement

This research employs machine learning models to detect anomalous network traffic using the UNSW-NB15 dataset Moustafa and Slay, 2016. The methodology includes data preprocessing, feature selection, and a comparative evaluation across six models. Standard performance metrics are applied to determine the most effective intrusion detection approach for strengthening cybersecurity.

1.5. Research Questions

The investigation aims to address the following central questions:

1. How can machine learning methods be effectively utilized on the UNSW-NB15 dataset to enhance the accuracy of network anomaly detection?
2. What approaches can reduce the number of false positives in anomaly detection systems while keeping the detection rates high?

3. How well can systems that find anomalies handle cyber threats that are always changing in networks that are complex and fluid?

1.6. Goals and Objectives

The central purpose of this work is to fulfill the following aims:

1. Apply a machine learning-centered examination of the UNSW-NB15 dataset, concentrating on appraising different models for identifying network anomalies.
2. Using and apply techniques to lower false positive rates in anomaly detection systems, therefore strengthening their dependability and accuracy.
3. Build a scalable and flexible anomaly detection framework able to react efficiently to new cybersecurity challenges in modern network environment.

2. CHAPTER II: LITERATURE REVIEW

2.1. Literature Review

Undertaking a thorough survey of prior work within a focused area is a vital prerequisite for scholars who wish to deliver meaningful progress. This task assists in constructing a robust theoretical underpinning for the intended investigation. The literature analysis offered in this segment draws attention to significant gaps that warrant deeper exploration. In the last few years, machine learning (ML) methods have attracted broad interest as potent instruments for addressing difficulties across numerous domains, including cybersecurity—notably in anomaly detection Abdulhammed et al., 2019 as well as in computer vision and behavioural analytics Z. Jiang et al., 2018. The fields in which machine learning is employed are growing swiftly, providing efficient answers to ever more complicated problems that span multiple disciplines Portugal et al., 2018. Although deep learning techniques offer strong capabilities, their reliance on large-scale data collections can occasionally restrict their practical deployment. As pointed out by Parampottupadam and Moldovann, 2018, conventional machine learning approaches can, in specific circumstances, outperform deep learning models. Machine learning techniques rest on statistical and mathematical frameworks that permit systems to generate predictions from data with little human involvement. These methods are usually divided into supervised, unsupervised, semi-supervised, and reinforcement learning, and they have demonstrated notable effectiveness in managing vast and intricate datasets. Several integrated and hierarchical methodologies have been proposed to improve intrusion detection systems (IDS). A hybrid method combining Genetic Algorithms with Least Squares Support Vector Machines was designed by Gharaee and Hosseinvand, 2016, where feature subsets are represented as chromosomes, resulting in robust classification performance. In the work ofSalman et al., 2017, Random Forest was applied for dimensionality reduction on the UNSW-NB15 dataset to classify eight attack types, with performance assessed using metrics including False Alarm Rate and Undetected Rate.Sharma et al., 2019 introduced a multi-layered detection framework that utilizes an Extreme Learning Machine ensemble for classification and Extra Trees for feature ranking, followed by a soft decision fusion layer to enhance overall accuracy.

Khammassi and Krichen, 2017 combined a Genetic Algorithm for feature selection with Logistic Regression and tested the approach on the UNSW-NB15 and KDDCup99 datasets. Coupled with a Decision Tree classifier, the GA-LR method achieved detection rates of 81.42% and 99.90% respectively, while also reducing false alarm rates. Moukhafi et al., 2019 applied a Genetic Algorithm to eliminate redundant features and used a Self-Organizing Map for classification, resulting in improved detection. Likewise, Ren et al., 2019 integrated Genetic Algorithm-based feature selection with Random Forest and

Isolation Forest for resampling, thereby ensuring high accuracy and low false positives.

Feature reduction approaches have been shown to achieve higher accuracy compared to models utilizing the complete set of features. As an illustration, Manjunatha et al., 2019 used an SVM classifier for multiclass classification following feature selection based on Mutual Information and Linear Correlation Coefficient (MI-LCC).

K. Jiang et al., 2020 addressed class imbalance by applying oversampling (SMOTE) together with undersampling (One-Sided Selection), subsequently extracting features through Convolutional Neural Networks and feeding them into a Bidirectional Long Short-Term Memory network. Sethi et al., 2020 put forward an intrusion detection system that uses deep reinforcement learning for classification and chi-square feature selection, with true positive rate, false positive rate, and accuracy derived from ROC curve analysis for performance assessment.

Wu et al., 2020 introduced Densely-ResNet, a deep architecture composed of multiple Conv-GRU modules linked by dense connections. The model was built to detect external cyberattacks across fog, edge, and cloud computing layers using the ToN-IoT dataset. Sarhan et al., 2021 carried out a detailed examination of network attack attributes, assessing their significance and predictive power. Features were ranked using chi-squared, correlation, and information gain, and subsequently tested with random forest and a deep feedforward network for attack classification. Moustafa, 2021 introduced an evaluation framework designed for the ToN-IoT dataset to benchmark artificial intelligence-based security systems. A dynamic architecture incorporating fog, edge, and cloud layers was implemented using NSX vCloud NFV, and four machine learning models—random forest, naive Bayes, deep neural networks, and gradient boosting machines—were used for the evaluation.

In addition, Ferrag et al., 2022 presented the Edge-IIoT dataset to facilitate experimental research on intrusion detection. This dataset enabled both centralized assessment and federated deep learning experiments. Three classification tasks—binary, six-class, and fifteen-class were investigated. Four classifiers, namely random forest, support vector machine, k-nearest neighbors, and deep neural networks, were applied to evaluate various types of cyberattacks.

2.2. Summary of Literature Review

Reference	Approach	Key Findings	Limitations
Portugal et al., 2018	Review of machine learning techniques	Increasing integration of ML methods across a wide range of application areas	Deep learning models often require large datasets, which can restrict their practical use in some scenarios
Parampottupadam and Moldovann, 2018	Comparative study between deep learning and traditional ML approaches	Showed that classical ML models can outperform deep learning in certain problem settings	Conventional ML methods may provide better efficiency in specific applications compared to deep learning models
Gharaee and Hosseinvand, 2016	Hybrid intrusion detection system combining Genetic Algorithm (GA) and SVM	Improved True Positive Rate (TPR) performance achieved	Experiments were limited to a single dataset (UNSW-NB15)
Salman et al., 2017	Stepwise detection approach using Random Forest (RF)	Evaluated using False Alarm Rate (FAR) and Undetected Rate (UND) metrics	Excluded the fuzzers attack class from evaluation
Sharma et al., 2019	Multi-layer IDS using Extreme Learning Machine ensemble	Achieved higher accuracy through a hierarchical detection framework	Did not evaluate computational cost or real-time applicability
Khammassi and Krichen, 2017	GA-based feature selection with Logistic Regression and Decision Tree classifier	Achieved detection rates of 81.42% (UNSW-NB15) and 99.90% (KDDCup99), along with reduced false alarms	Performance varies depending on dataset characteristics
Moukhafi et al., 2019	Genetic Algorithm-based feature selection with SOM classifier	Improved IDS performance through optimized feature selection	Strong dependence on GA optimization process
Ren et al., 2019	GA and Random Forest for feature selection combined with Isolation Forest	Delivered high accuracy and reduced false positive rates	High computational complexity may limit real-time deployment
Manjunatha et al., 2019	MI-LCC feature selection with SVM classifier	Improved classification accuracy by selecting relevant features and reducing noise	Risk of overfitting due to dataset-specific tuning

K. Jiang et al., 2020	CNN-based spatial feature extraction with BiLSTM temporal modeling	Used resampling methods to address class imbalance, improving classification results	Deep learning models require high computational resources
Sethi et al., 2020	Chi-square feature selection combined with deep reinforcement learning	Achieved improved performance validated using ROC analysis	High computational cost and need for further evaluation in cloud-based environments
Wu et al., 2020	Densely-ResNet architecture with Conv-GRU layers	Achieved strong detection capability using fog, edge, and cloud integration on ToN-IoT dataset	Requires large-scale datasets and real-time deployment feasibility is uncertain
Sarhan et al., 2021	Feature selection using chi-squared, correlation, and information gain with RF and deep networks	Improved detection performance through optimized feature selection and classifier fusion	Potential overfitting and dependency on dataset distribution
Moustafa, 2021	Testing framework for ToN-IoT using NSX vCloud NFV with multiple ML models	Enabled multi-layer evaluation across fog, edge, and cloud environments	Complex setup and limited validation across diverse real-world networks
Ferrag et al., 2022	Edge-IIoT dataset supporting centralized and federated IDS evaluation	Conducted binary, six-class, and fifteen-class classification using multiple classifiers	Limited generalization beyond Edge-IIoT dataset and challenges in real-time federated learning

3. CHAPTER III: METHODOLOGY

The present study employs the UNSW-NB15 dataset, which encompasses nine distinct categories of cyberattacks, to benchmark a range of machine learning techniques for intrusion detection. The workflow consists of important steps, such as preprocessing the data, extracting attributes, and training the classifier. Then, the models' predictive performance is compared. Figure 3.1 shows how this experiment was set up. To find the best way to check an organization's security readiness and help set up the best intrusion detection system, the models are tested against important performance measures like accuracy, precision, recall, and F1-score.

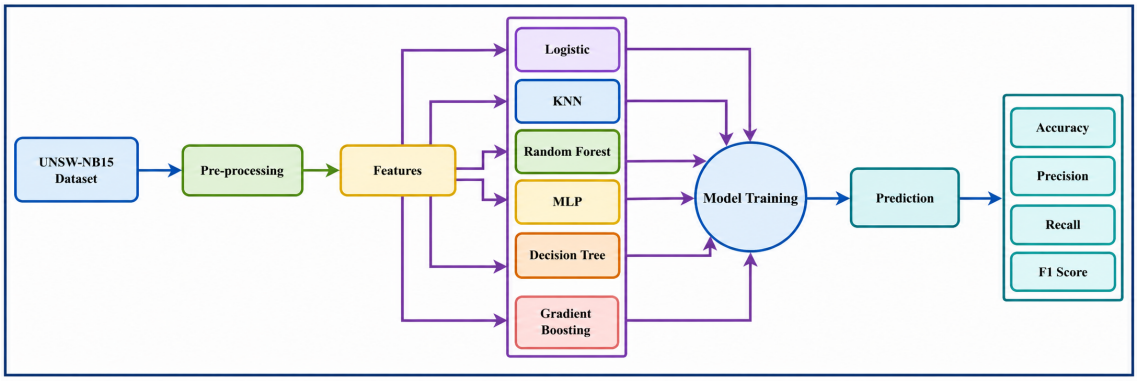

Fig. 3.1. Research Overview Block diagram

3.1. Dataset

The UNSW-NB15 dataset serves as a commonly used benchmark in network intrusion detection, combining genuine network traffic with artificially generated attacks. Developed by the Australian Centre for Cyber Security (ACCS) using the IXIA Perfect Storm tool, the dataset includes 49 features and nine types of attacks, derived from 100 GB of raw network data. It has become a standard reference in cybersecurity research, particularly for machine learning-based anomaly detection. The dataset is divided into 175,341 training instances and 82,332 testing instances Moustafa and Slay, 2015.

3.2. Logistic Regression Model

Logistic regression is a supervised learning method designed for binary classification tasks. Despite its name, it addresses classification problems rather than regression. It relies on the sigmoid (logistic) function to derive class probabilities Hotzy et al., 2018. The model calculates a weighted sum of the input features and then applies the sigmoid

function to yield a result between 0 and 1, which indicates the likelihood of belonging to the positive class. Parameter estimation is carried out via Maximum Likelihood Estimation, which minimizes the binary cross-entropy (logistic) loss. The algorithm is widely applied in spam filtering, medical diagnosis, and credit risk evaluation Zaidi and Al Luhayb, 2023.

3.2.1. Mathematical Basis of Logistic Regression

For a feature vector $X = [x_1, x_2, ..., x_n]$, the model first calculates a linear combination:

$$z = w_0 + \sum_{i=1}^{n} w_i x_i$$

where w_i represent the model weights and w_0 is the bias term. The probability of belonging to the positive class is derived by passing this linear output through the sigmoid function:

$$P(y = 1|X) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

In the context of binary classification, the cost function is defined as the negative log-likelihood, also known as binary cross-entropy:

$$L(w) = -\sum_{i=1}^{m} [y_i \log P(y_i) + (1 - y_i) \log(1 - P(y_i))]$$

The optimal weight values are determined using gradient descent, where the parameters are updated iteratively as follows:

$$w_j^{(t+1)} = w_j^{(t)} - \eta \frac{\partial L}{\partial w_j}$$

In this expression, η stands for the learning rate. This iterative update process continues until the loss function reaches a minimum.

3.3. K-Nearest Neighbors (KNN)

KNN is an intuitive yet powerful machine learning method suited to both classification and regression problems. It falls under instance-based learning, where decisions are made by evaluating the similarity between data points S. Zhang et al., 2017. In a classification setting, a query sample is labeled with the class that appears most frequently among its k closest neighbors. When used for regression, the output is derived by computing the mean (or a weighted mean) of the target values of those neighboring points. The concept of distance is fundamental to the algorithm, with typical choices being Euclidean, Manhattan,

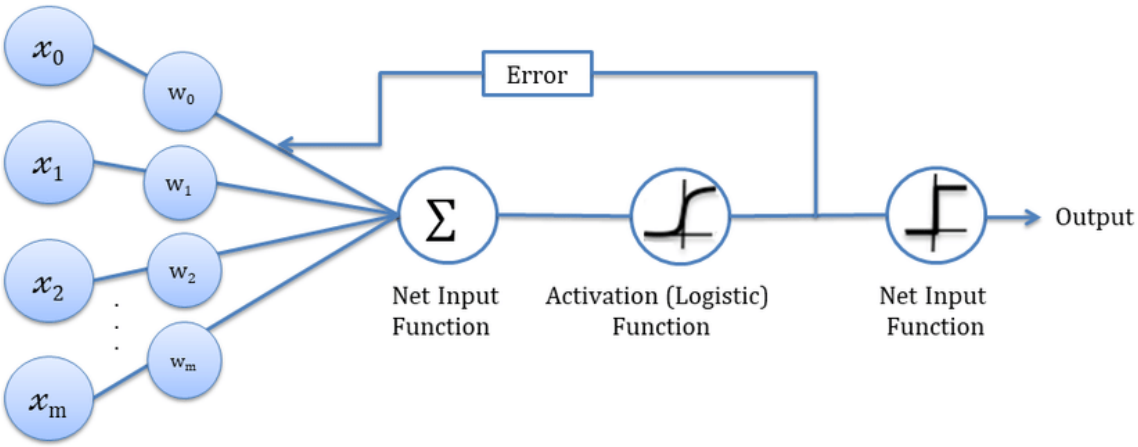

Fig. 3.2. Logistic regression model architecture Nobel et al., 2024

and Minkowski distances. While KNN is straightforward to grasp and implement, it can incur high computational costs for large datasets, as each prediction requires computing distances to all training instances Cunningham and Delany, 2021.

3.3.1. Mathematical Formulation of KNN

For a given input X, the distance between this point and each training example X_i is computed using a distance measure, most commonly the **Euclidean distance:**

$$d(X, X_i) = \sqrt{\sum_{j=1}^{n} (X_j - X_{i,j})^2}$$

where X_j and $X_{i,j}$ denote the feature values of the test sample and the training sample, respectively.

In classification tasks, the predicted class label $\hat{y}$ is determined by a majority vote among the k nearest neighbors:

$$\hat{y} = \arg\max_{c} \sum_{i \in N_k} \mathbb{1}(y_i = c)$$

Here, y_i is the class label of the i-th neighbor, $\mathbb{1}(\cdot)$ represents an indicator function, and N_k is the set of the k closest neighbors.

3.4. Decision Tree

A decision tree is a supervised machine learning method applied to both classification and regression tasks. It operates by recursively partitioning the dataset into increasingly smaller subsets based on the feature values that provide the most effective separation of

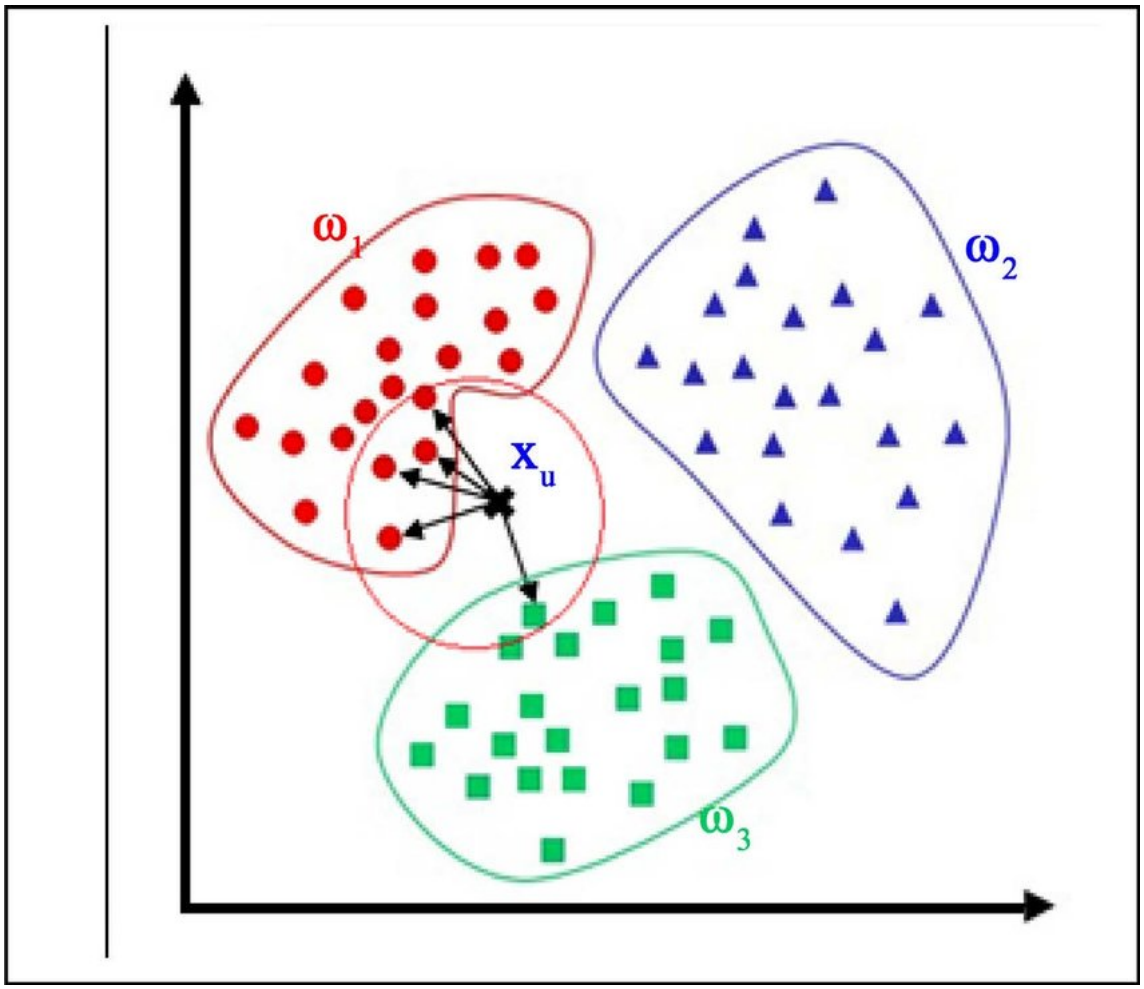

Fig. 3.3. illustration-of-K-nearest-neighbor-model W. Zhang et al., 2017

the data. This process repeats until the smallest possible groups have the highest possible purity with respect to the target variable. In classification trees, split selection criteria include Gini Impurity or Information Gain, whereas regression trees typically use Mean Squared Error (MSE). In the tree structure, internal nodes represent rules based on threshold values of features, and leaf nodes provide the final predictions (class label or numeric value). Tree growth ceases when certain termination criteria are met, such as a maximum depth or a minimum number of samples in a node. Decision Trees are highly interpretable and easy to use but can easily overfit the data without adequate pruning techniques Belgacem et al., 2018.

3.4.1. Mathematical Basis of Decision Trees

The primary objective when constructing a decision tree is to select splits that reduce impurity within the data. For classification problems, impurity is often measured using the Gini Impurity index, defined as:

$$Gini(t) = 1 - \sum_{i=1}^{k} p_i^2 \tag{3.1}$$

where p_i represents the proportion of samples belonging to class i in node t, and k is the total number of classes. The algorithm chooses the feature that provides the largest reduction in impurity, measured by **Information Gain**:

$$IG(t, f) = Gini(t) - \sum_{v \in \{v_1, v_2, \ldots, v_m\}} \frac{|t_v|}{|t|} Gini(t_v) \tag{3.2}$$

Here, t_v denotes the child nodes resulting from splitting node t based on feature f at its different values v, and $|t_v|$ is the number of instances in each child subset.

For regression tasks, impurity is evaluated using the Mean Squared Error (MSE):

$$MSE(t) = \frac{1}{|t|} \sum_{i=1}^{|t|} (y_i - \hat{y})^2 \tag{3.3}$$

In this expression, $\hat{y}$ is the mean target value within node t. The tree building process repeatedly, at each step, selects the feature and split point that maximize Information Gain (for classification) or minimize MSE (for regression), depending on the task at hand.

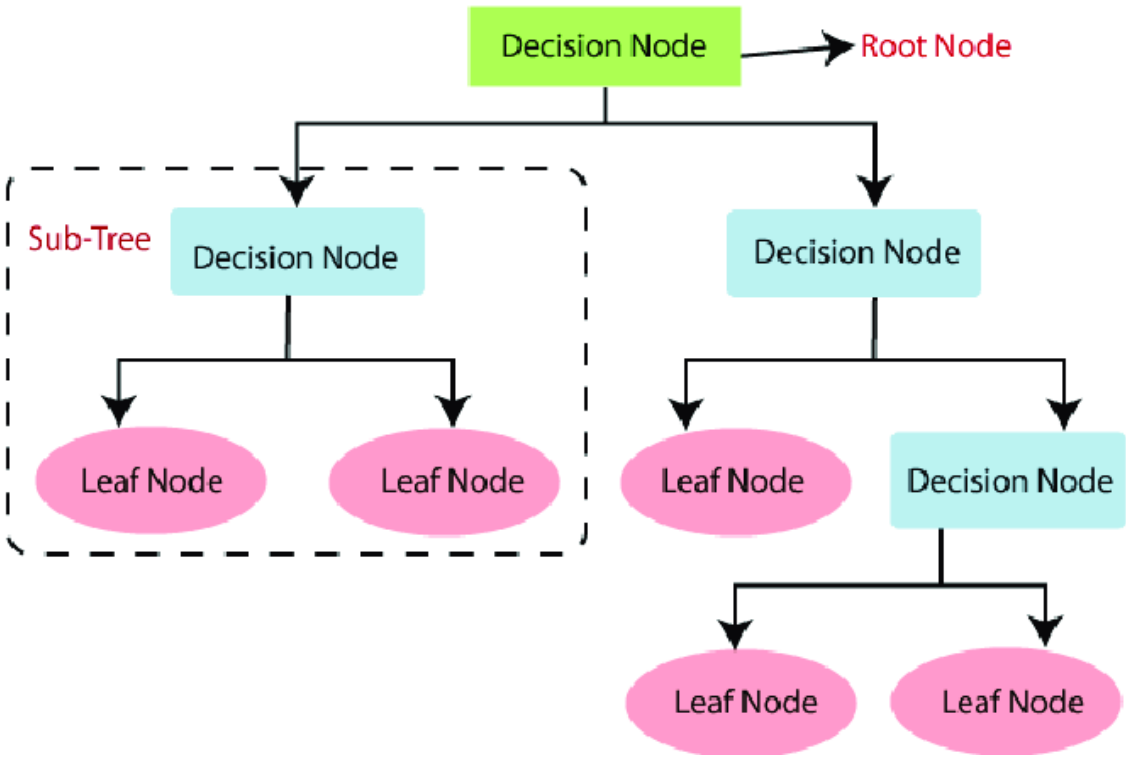

Fig. 3.4. Structure-of-the-decision-tree-DT Hafeez et al., 2021

3.5. Random Forest

Random Forest is an ensemble method composed of multiple decision trees, where each tree is trained on a bootstrap sample and uses a randomly selected subset of features at every split, making the approach applicable to both classification and regression problems Jaiswal and Samikannu, 2017. In classification tasks, the final output is determined by majority voting across all trees, whereas in regression, the output is the average of the individual tree predictions. Given a dataset D, the algorithm generates T trees from separate bootstrap samples D_t. Each tree produces a prediction $\hat{y}_t$, and the ensemble result is either the mode (for classification) or the mean (for regression) of these predictions. At each node, splitting is limited to a randomly chosen set of m features out of the total p features to encourage diversity among the trees. Split quality is evaluated using Gini impurity or entropy for classification, and Mean Squared Error for regression.

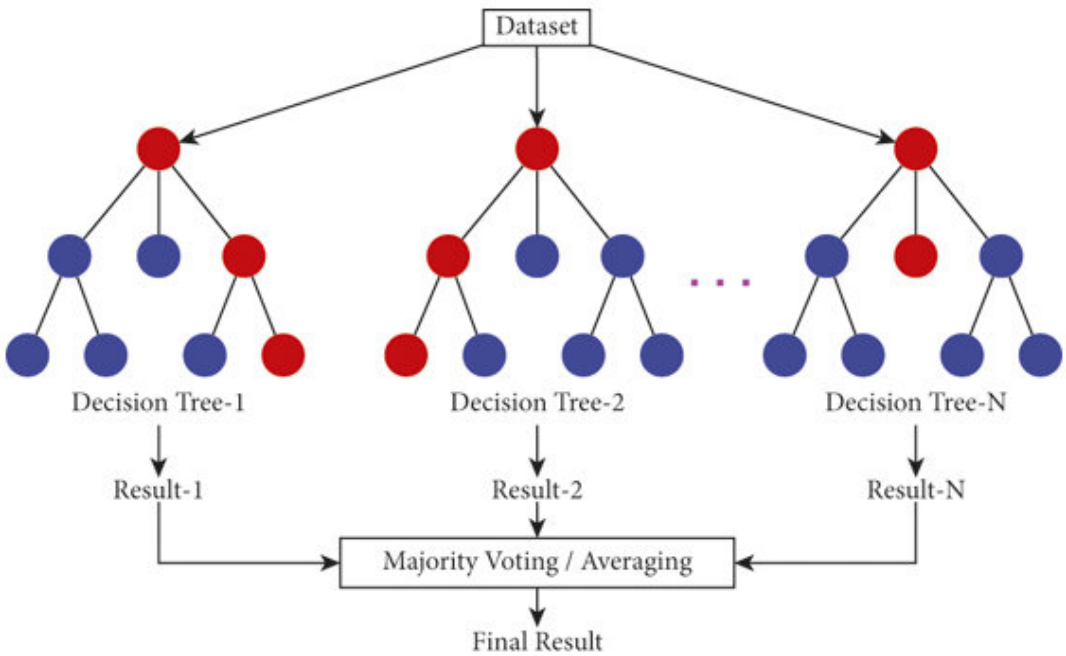

Fig. 3.5. Depiction of a Random Forest tree ensemble Khan et al., 2021

3.6. Multi-Layer Perceptron (MLP)

Multi-Layer Perceptrons (MLPs) represent a class of feedforward neural networks consisting of an input layer, at least one hidden layer, and an output layer. They are capable of learning complex patterns in data through weighted connections among neurons. The inclusion of nonlinear activation functions—such as ReLU, sigmoid, or tanh—allows them to model intricate relationships and enhances their expressive power. Training of the network is carried out using the backpropagation algorithm, where error signals are transmitted backward through the network, and the weights are adjusted using optimization

techniques like gradient descent or its adaptive variants (e.g., Adam, RMSprop). Because they can model highly nonlinear mappings, MLPs are extensively employed in both classification and regression scenarios Riedmiller and Lernen, 2014.

3.6.1. Mathematical Formulation

Given an input feature vector $X = [x_1, x_2, ..., x_n]$, the linear combination received by a neuron in a hidden layer is expressed as:

$$z_j = \sum_{i=1}^{n} w_{ij} x_i + b_j$$

where w_{ij} denotes the weight connecting the i-th input to the j-th neuron, and b_j is the bias term. The neuron then applies an activation function $f(\cdot)$ to this weighted sum to generate its output:

$$a_j = f(z_j)$$

This forward pass is performed sequentially across layers, with the activations from one layer serving as inputs to the subsequent layer. At the output layer, the network's prediction $\hat{y}$ is obtained as:

$$\hat{y} = f\left(\sum_{j=1}^{m} w_{jo} a_j + b_o\right)$$

Here, m is the number of neurons in the final hidden layer. The training procedure minimizes a loss function $L(y, \hat{y})$ via the backpropagation algorithm. For regression tasks, the Mean Squared Error (MSE) is frequently used, while cross-entropy is commonly adopted for classification. The weights are updated iteratively using the following rule:

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \frac{\partial L}{\partial w_{ij}}$$

In this expression, η represents the learning rate, which controls the step size of the weight updates and progressively reduces the prediction error.

3.7. Gradient Boosting

Gradient Boosting is an ensemble method for classification and regression that builds models sequentially, with each new learner correcting the residuals of the previous ones Natekin and Knoll, 2013. It adds shallow decision trees stage-wise, using gradient descent to minimize a differentiable loss. At each iteration, a tree is fitted to the negative

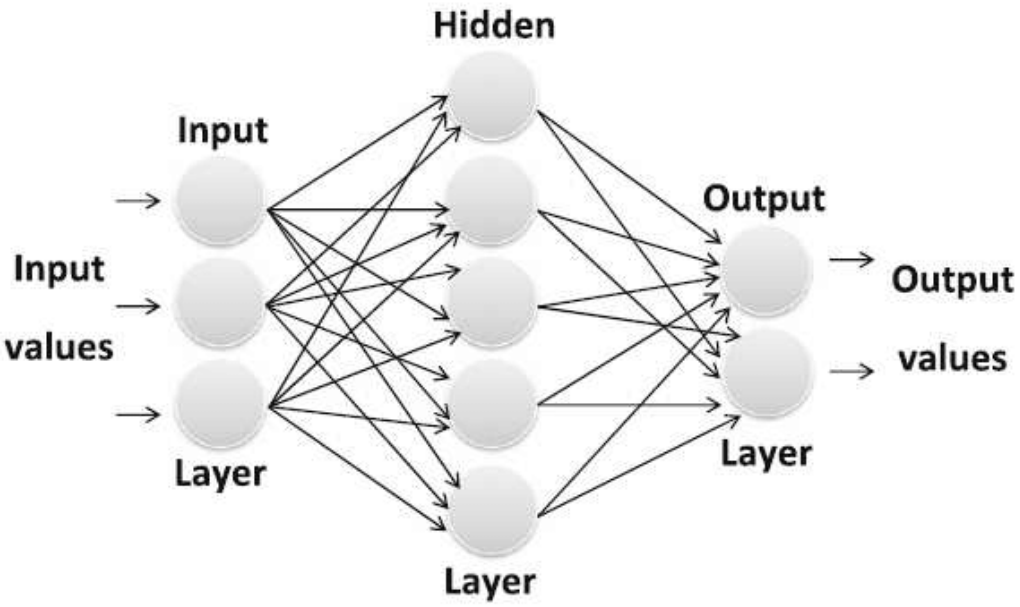

Fig. 3.6. MultiLayer-Perceptron-Classifier Al-Naymat et al., 2016

gradient (pseudo-residuals) of the loss with respect to the current ensemble prediction, progressively reducing bias. Performance depends on the learning rate, tree depth, and number of boosting rounds.

3.7.1. Mathematics of Gradient Boosting

Given $D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, the algorithm constructs $F(x)$ to minimize $L(y, F(x))$. The initial model is a constant minimizing the empirical loss (often the mean of the targets for regression):

$$F_0(x) = \arg\min_c \sum_{i=1}^{n} L(y_i, c) \tag{3.4}$$

At step m, pseudo-residuals are computed as the negative gradient:

$$r_i^{(m)} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)} \tag{3.5}$$

A weak learner (e.g., a shallow tree) is then fitted to these residuals:

$$h_m(x) = \arg\min_h \sum_{i=1}^{n} (r_i^{(m)} - h(x_i))^2 \tag{3.6}$$

The model is updated with a scaled version of the new learner:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x) \tag{3.7}$$

The learning rate η shrinks the contribution of each tree. The procedure repeats for M iterations. Common loss functions include mean squared error for regression and log loss for classification.

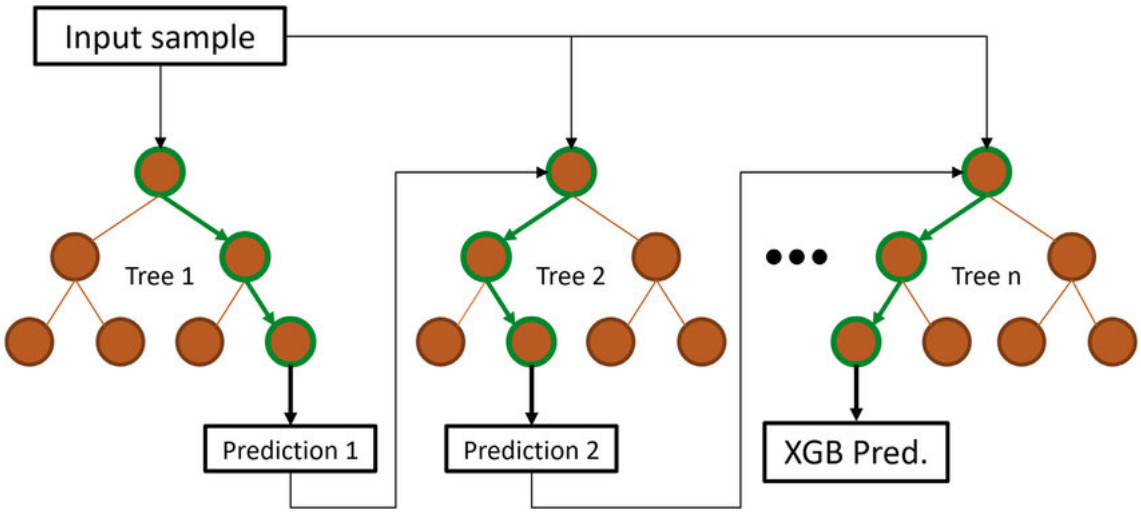

Fig. 3.7. Gradient-boosting-principleTausendschön et al., 2023

3.8. ReLU (Rectified Linear Unit)

The Rectified Linear Unit, commonly referred to as ReLU, is a widely adopted activation function in neural networks. It outputs the input directly if the input is positive, and returns zero for negative inputs. This simple operation introduces essential non-linearity into the network while also accelerating the training process Kessler et al., 2017. ReLU is mathematically expressed as:

$$\mathrm{ReLU}(x) = \max(0, x) \tag{3.8}$$

3.9. Adam (Adaptive Moment Estimation)

Adam is a prevalent optimization algorithm in deep learning that adjusts network weights efficiently. It enhances the parameter update by combining momentum—the exponentially decaying average of previous gradients—with adaptive learning rates tailored to each parameter Yi et al., 2020.

3.10. Performance Metrics

3.10.1. Accuracy

Accuracy is the overall ratio of correctly classified instances (both positive and negative).

Formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

3.10.2. Precision

Precision measures the reliability of positive predictions, i.e., the proportion of true positives among all predicted positives.

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

3.10.3. Recall

Recall (sensitivity) quantifies the proportion of actual positive cases correctly identified by the model.

Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

3.10.4. F1-Score

The F1-score is the harmonic mean of precision and recall, delivering a balanced measure useful for imbalanced datasets.

Formula:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

4. CHAPTER 4: EXPERIMENTAL OUTCOMES

This section compares eight machine learning classifiers: Logistic Regression, k-Nearest Neighbors (kNN), Decision Tree, Random Forest, Gradient Boosting, Multi-Layer Perceptron (MLP), Support Vector Machine (SVM), and XGBoost, based on how well they can make predictions and how quickly they can do so. The test uses a number of different measures, such as accuracy, precision, recall, F1-score, ROC-AUC, Matthews Correlation Coefficient (MCC), and Cohen's Kappa. To find out how efficient something is, you keep track of how long it takes to train and how long it takes to make a prediction. The goal is to find the best way to detect network intrusions in real time, balancing speed and accuracy. The results are put into a table, then turned into a graph, analyzed, ranked, and finally checked to see how useful they are in cybersecurity.

4.1. Logistic Regression Model Assessment

The Logistic Regression model achieved an accuracy of 92.56%, demonstrating its effectiveness in separating benign and malicious traffic. The recall of 92.56% indicates that the model identifies a high proportion of true positives (attack), while the precision of 92.58% suggests most positive results are correct. The F1-score of 92.56% confirms a good combination of precision and recall, and stable performance shown in Table.4.1 and 4.2. The confusion matrix is shown in Fig.4.1. Additionally, a ROC-AUC of 0.9257 indicates good discrimination between attack and normal traffic. The MCC (0.8500) and Cohen's Kappa (0.8498) values confirm a close match between predicted labels and the ground truth, providing a more reliable measure than accuracy alone. When it comes to time efficiency, the training time is just 1.85 seconds, and inference time (time to make predictions) is next to nothing (0.00s), making this model an efficient option for intrusion detection.

Table 4.1. Key performance measurements and runtime details for the logistic regression classifier.

Classifier	**Acc.**	**Rec.**	**Prec.**	**F1**	**AUC**	**MCC**	κ	**Train (s)**	**Pred (s)**	**Total (s)**
Logistic Regression	92.56%	92.56%	92.58%	92.56%	0.9257	0.8500	0.8498	1.85	0.00	1.85

4.1.1. Per-Class Evaluation

Table 4.2. Class-level precision, recall, and F1-score breakdown for Logistic Regression.

Traffic Type	Precision	Recall	F1-Score	Sample Count
Normal	0.91	0.93	0.92	3700
Attack	0.94	0.92	0.93	4534
Accuracy		0.93		8234
Macro Average	0.92	0.93	0.92	8234
Weighted Average	0.93	0.93	0.93	8234

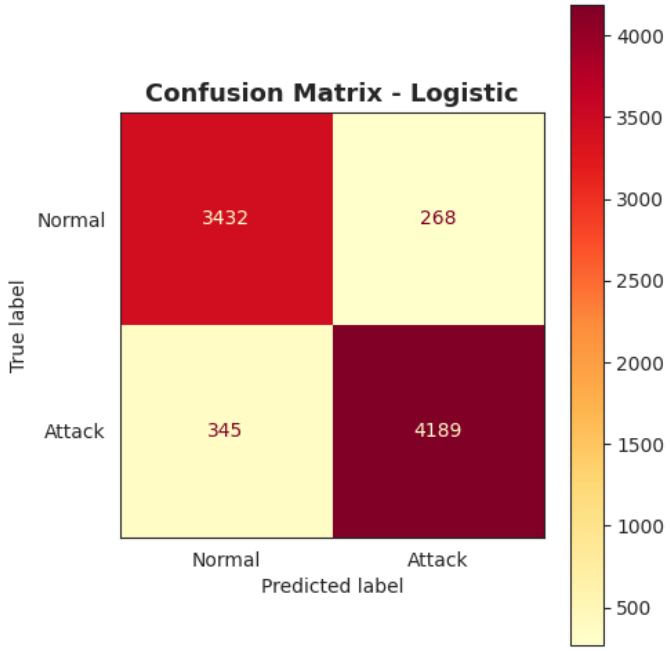

Fig. 4.1. Logistic Regression model confusion matrix

The findings confirm that the k-Nearest Neighbors classifier delivers a powerful ability to spot cyber threats, generating consistently high marks across the full range of evaluation criteria.

4.2. k-Nearest Neighbors Classifier

The KNeighborsClassifier model was intelligent to correctly identify network intrusion patterns 94.84% of the time, which demonstrates that it is a consistent model. The 94.84% recall shows that the algorithm catches a lot of attacks, and the 94.95% precision illustrates that almost all of its positive predictions are right. The F1-score of 94.85% shows that the recall and precision are equal, as shown in the Table.4.3 and 4.4.The confusion

matrix is shown in Fig.4.2 With an Area Under the Curve (ROC-AUC) score of 0.9501, it is very good at effective the difference between good and bad traffic. Also, a Matthews Correlation Coefficient (MCC) of 0.8971 and a Cohen's Kappa of 0.8962 show that the predicted and actual labels are very close to each other, which shows that the model is reliable. Turning to the model's competence, it trains in an insignificant time (0.01 seconds). Contrariwise, the prediction phase takes a relatively long 6.17 seconds, implying that while training is a light process, predictions can be calculation-heavy, which could limit their use in time-sensitive situations.

Table 4.3. Performance indicators and timing results for the kNN model.

Algorithm	**Acc.**	**Rec.**	**Prec.**	**F1**	**AUC**	**MCC**	κ	**Train (s)**	**Pred (s)**	**Total (s)**
KNeighborsClassifier	94.84%	94.84%	94.95%	94.85%	0.9501	0.8971	0.8962	0.01	6.17	6.18

Table 4.4. Precision, recall, and F1-score per traffic category for the kNN classifier.

Traffic Class	**Precision**	**Recall**	**F1-Score**	**Samples**
Normal	0.92	0.97	0.94	3700
Attack	0.97	0.93	0.95	4534
Overall Accuracy		0.95		8234
Macro Average	0.95	0.95	0.95	8234
Weighted Average	0.95	0.95	0.95	8234

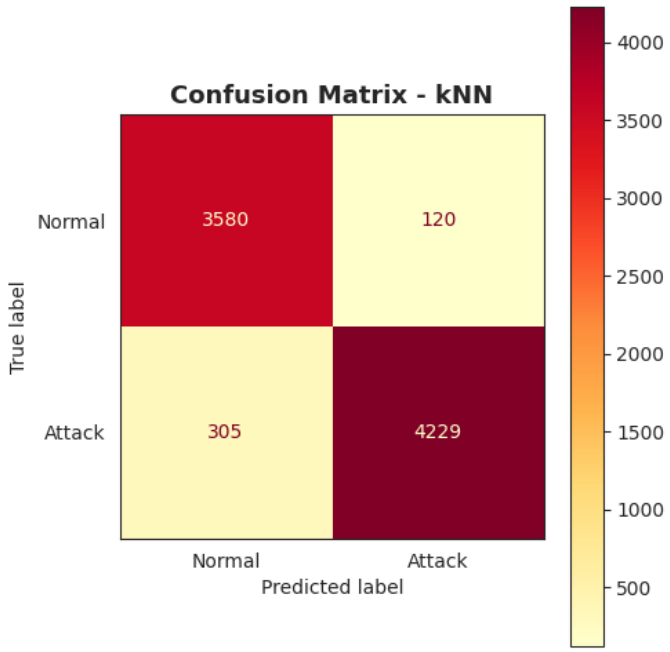

Fig. 4.2. KNeighbors Classifier model confusion matrix

This evaluation reveals that while the KNeighborsClassifier attains high accuracy and robust overall metrics, its notably longer inference time reduces its efficiency compared to many other algorithms.

4.3. Decision Tree Classifier

The Decision Tree model attained an accuracy of 96.68%, showing a high competence to successfully identify network intrusions. The recall of 96.68% demonstrates that the classifier possesses the ability to identify most of the true attack proceedings, while the precision of 96.68% designates that the positive predictions are extremely reliable, as shown in the Table4.5 and 4.6. The F1-score of 96.68% (harmonic mean of precision and recall) reinforces its stable performance. The confusion matrix is shown in Fig.4.3. The ROC-AUC of 0.9665 indicates a high aptitude to distinguish between normal and attack traffic. Also, both the Matthews Correlation Coefficient (MCC) and Cohen's Kappa are at 0.9330, which designates a very close alignment between the predicted and the true labels, and underlines the model's reliability. In terms of computational efficiency, the training phase takes a reasonable 1.44 seconds, and the prediction phase an almost immediate 0.00s. These topographies render the Decision Tree very appropriate for real-time intrusion detection systems, where quick, in-the-moment decisions are compulsory.

Table 4.5. Performance statistics and timing measurements for the Decision Tree classifier.

Classifier	**Acc.**	**Rec.**	**Prec.**	**F1**	**AUC**	**MCC**	κ	**Train (s)**	**Pred (s)**	**Total (s)**
Decision Tree	96.68%	96.68%	96.68%	96.68%	0.9665	0.9330	0.9330	1.44	0.00	1.44

Table 4.6. Per-class precision, recall, and F1-score values for the Decision Tree model.

Traffic Type	**Precision**	**Recall**	**F1-Score**	**Samples**
Normal	0.96	0.96	0.96	3700
Attack	0.97	0.97	0.97	4534
Accuracy		0.97		8234
Macro Average	0.97	0.97	0.97	8234
Weighted Average	0.97	0.97	0.97	8234

This assessment confirms that the Decision Tree delivers high accuracy, instant predictions, and strong overall efficiency, making it highly appropriate for real-time network defence applications.

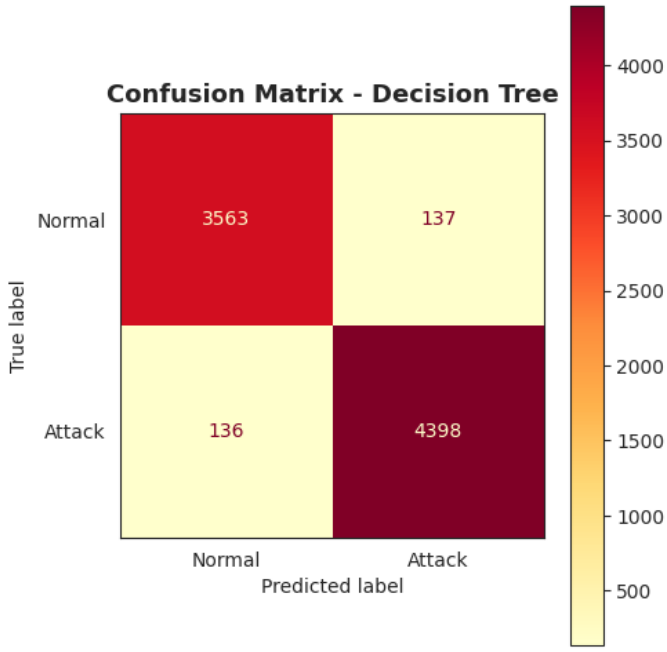

Fig. 4.3. Decision Tree model confusion matrix

4.4. Random Forest Classifier Evaluation

The Random Forest ensemble was an excellent model for finding network attacks, with an accuracy of 97.70%. The model's recall of 97.70% means that it finds almost all real attacks, and its precision of 97.71% means that its warnings are reliable. The F1-score of 97.71% shows that the relationship between precision and recall is well-balanced. Table 4.7, Table 4.8. The confusion matrix is shown in Fig.4.4. A ROC-AUC of 0.9771 means that it is very good at telling the difference between good and bad traffic. A Matthews Correlation Coefficient (MCC) of 0.9537 and a Cohen's Kappa of 0.9536 also show that the predicted labels and the actual labels are very similar, which shows that the classifier is reliable. But this performance comes at the cost of a long training time: it takes 72.80 seconds, and this is significantly greater than many other models. The inference process, on the other hand, takes only 0.45 seconds, so once trained, the model can respond reasonably quickly. This makes Random Forest an attractive option for scenarios where optimal accuracy is critical and prediction time is less critical.

4.4.1. Performance Overview

Table 4.7. Overall performance figures and timing breakdown for the Random Forest model.

Method	Acc.	Rec.	Prec.	F1	AUC	MCC	κ	Train (s)	Pred (s)	Total (s)
Random Forest	97.70%	97.70%	97.71%	97.71%	0.9771	0.9537	0.9536	72.80	0.45	73.25

Table 4.8. Precision, recall, and F1 measures for each traffic class using Random Forest.

Traffic Class	Precision	Recall	F1-Score	Sample Count
Normal	0.97	0.98	0.97	3700
Attack	0.98	0.98	0.98	4534
Accuracy		0.98		8234
Macro Average	0.98	0.98	0.98	8234
Weighted Average	0.98	0.98	0.98	8234

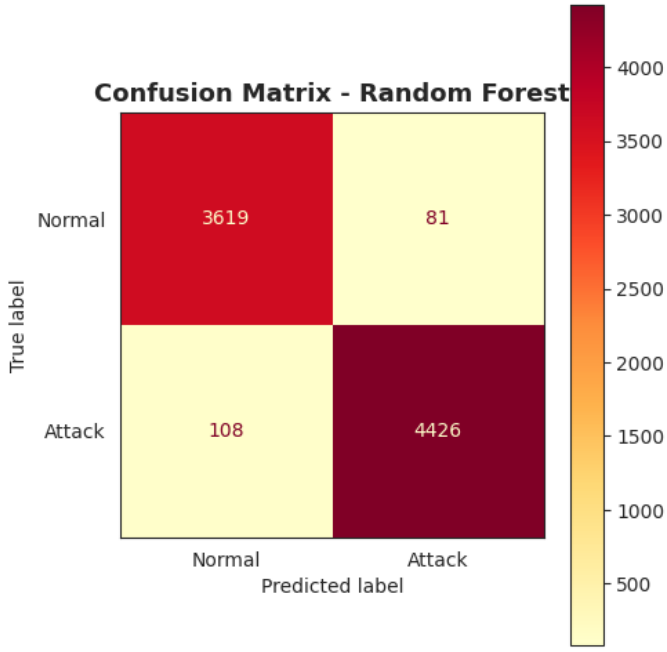

Fig. 4.4. Random Forest model confusion matrix

This evaluation demonstrates that the Random Forest model achieves very high accuracy and strong overall performance, making it a highly reliable option for intrusion detection systems, even though it requires longer training time compared to other models.

4.5. Gradient Boosting Classifier Analysis

The Gradient Boosting model was able to find network intrusions with 95.73% accuracy, which shows how good it is at it. The model's recall value of 95.73% means that it finds a lot of real attacks, and its precision value of 95.73% means that most of its positive predictions are correct. Its F1 score of 95.73% shows in Table.4.9 , Table. 4.10. it has a good balance of precision and recall. An ROC-AUC of 0.9572 means that it can tell the difference between normal and bad traffic very well. The Matthews Correlation Coefficient (MCC) and Cohen's Kappa are 0.9137, which shows a strong link between the predicted and true labels and shows that the model is reliable and valid. Training is fairly

costly, taking 48.21 seconds, but it is much faster than training a simpler classifier. The confusion matrix is shown in Fig.4.5 In contrast, inference is very quick, taking only 0.02 seconds. This suggests that once trained, the model becomes a very promising candidate for real-time intrusion detection systems.

Table 4.9. Performance indicators and execution times for the Gradient Boosting classifier.

Technique	**Acc.**	**Rec.**	**Prec.**	**F1**	**AUC**	**MCC**	κ	**Train (s)**	**Pred (s)**	**Total (s)**
Gradient Boosting	95.73%	95.73%	95.73%	95.73%	0.9572	0.9137	0.9137	48.21	0.02	48.23

Table 4.10. Precision, recall, and F1-score per traffic category for the Gradient Boosting model.

Traffic Type	**Precision**	**Recall**	**F1-Score**	**Sample Count**
Normal	0.95	0.96	0.95	3700
Attack	0.96	0.96	0.96	4534
Accuracy		0.96		8234
Macro Average	0.96	0.96	0.96	8234
Weighted Average	0.96	0.96	0.96	8234

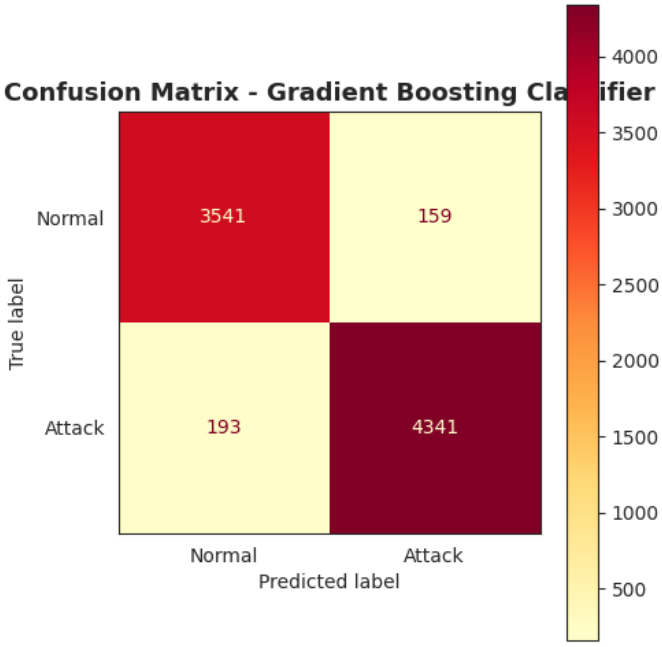

Fig. 4.5. Gradient Boosting model confusion matrix

These results demonstrate that the Gradient Boosting classifier achieves strong predictive accuracy and rapid inference, establishing it as a compelling choice for intrusion detection systems, albeit at the cost of a markedly longer training stage.

4.6. Multi-Layer Perceptron (MLP) Classifier

The Multi-Layer Perceptron (MLP) neural network was able to recognize network intrusion events with 96.67% accuracy, which shows that it is very good at doing so. With a recall rate of 96.67%, the model catches most real attacks, and with a precision rate of 96.67%, its positive predictions are very reliable. The F1-score of 96.67% shows that there is a good balance between precision and recall. Table4.11 and Table4.12. A ROC-AUC value of 0.9666 shows that it is very good at telling the difference between friendly and hostile traffic. The confusion matrix is shown in Fig.4.6 . The Matthews Correlation Coefficient (MCC) and Cohen's Kappa both equal 0.9328, which shows that the predicted and true labels are very close to each other and that the model is strong. The training phase takes 35.02 seconds, which is a lot of time compared to simpler classifiers. But the stage where you make an inference is very The inference stage, on the other hand, is very fast, taking only 0.01 seconds. This makes the model perfect for real-time intrusion detection settings where quick, on-the-fly decisions are needed.

Table 4.11. Aggregate performance metrics and execution times for the MLP classifier.

Method	**Acc.**	**Rec.**	**Prec.**	**F1**	**AUC**	**MCC**	κ	**Train (s)**	**Pred (s)**	**Total (s)**
MLP Classifier	96.67%	96.67%	96.67%	96.67%	0.9666	0.9328	0.9328	35.02	0.01	35.03

Table 4.12. Per-class precision, recall, and F1-score values for the Multi-Layer Perceptron (MLP) model.

Traffic Type	**Precision**	**Recall**	**F1-Score**	**Sample Count**
Normal	0.96	0.97	0.96	3700
Attack	0.97	0.97	0.97	4534
Aggregate Accuracy		0.97		8234
Macro Average	0.97	0.97	0.97	8234
Weighted Average	0.97	0.97	0.97	8234

The findings confirm that the Multi-Layer Perceptron neural network delivers reliable intrusion detection while keeping inference latency extremely low, rendering it well-suited for real-time cybersecurity deployments.

4.7. Model Comparison and Findings

Table 4.13 shows an overall method of how well each method works. An in-depth analysis of the six algorithms across accuracy, recall, precision, F1-score, ROC-AUC, MCC, and Cohen's Kappa showed that Random Forest was the best predictor (97.70% accuracy,

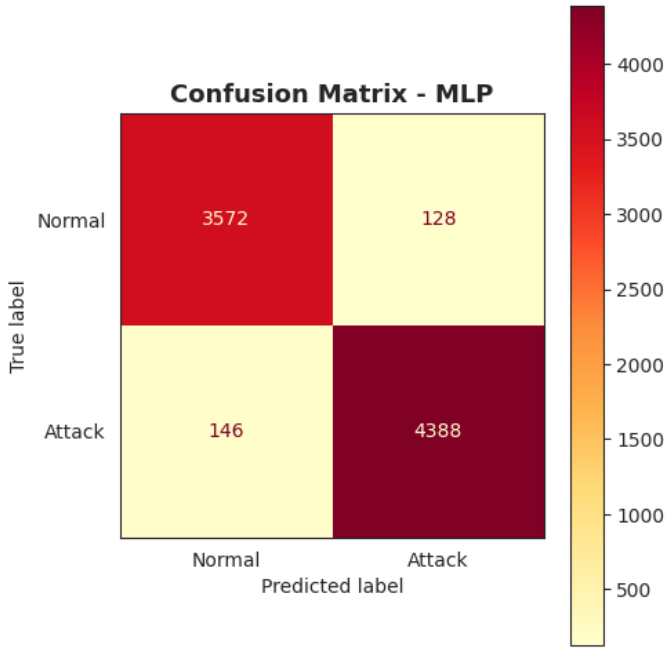

Fig. 4.6. Confusion matrix of the Neural Network MLP model

0.9771 ROC-AUC, 0.9537 MCC) but the most expensive (72.80 s training time). Decision Tree (96.68% accuracy) and MLP (96.67% accuracy) achieved nearly identical detection performance with significantly reduced total times (1.44 s and 35.03 s, respectively) and near-real-time inference, making them the best choices for real-time network intrusion detection. Gradient Boosting (95.73% accuracy) and k-Nearest Neighbors (94.84% accuracy) achieved similar but inferior performance, with kNN suffering from slow predictions (6.17 s) and Gradient Boosting taking longer to train without matching the accuracy of tree-based algorithms. Also, in shown the results in Fig.4.7. Logistic Regression were the quickest (1.85 s total) but produced the poorest results (92.56% accuracy) and is only suitable for baseline comparisons. In summary, Random Forest is ideal for achieving superior detection performance when time is not a concern, while Decision Tree and MLP provide an outstanding time-performance trade-off for resource-limited real-time intrusion detection systems.

Table 4.13. Performance measures across the six evaluated classifiers.

Classifier	**Accuracy**	**Recall**	**Precision**	**F1-Score**
Logistic Regression	92.56%	92.56%	92.58%	92.56%
k-NN	94.84%	94.84%	94.95%	94.85%
Decision Tree	96.68%	96.68%	96.68%	96.68%
Random Forest	97.70%	97.70%	97.71%	97.71%
Gradient Boosting	95.73%	95.73%	95.73%	95.73%
MLP	96.67%	96.67%	96.67%	96.67%

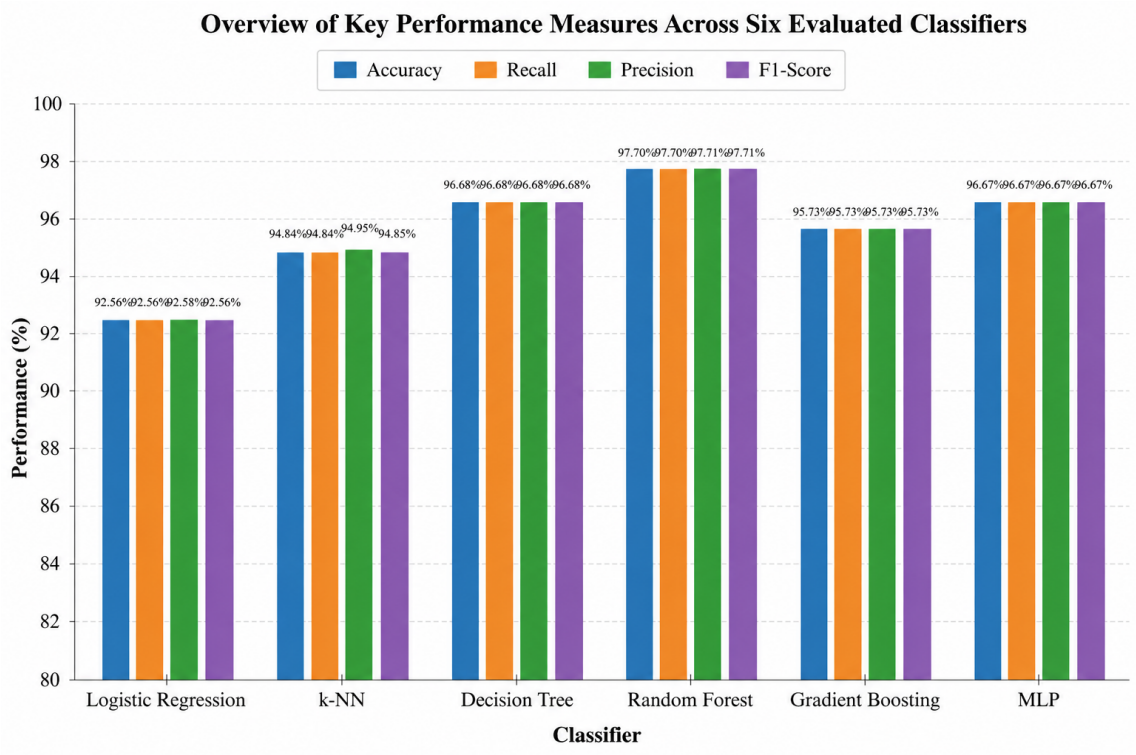

Fig. 4.7. Models Performance Metrics

4.8. Model Training and Prediction Time Analysis

The performance of a range of machine learning algorithms (such as SVM and XGBoost) was evaluated in terms of training time, prediction time and total time. Table **??**. These metrics are also represented graphically in Figure 4.8. Trained and predicted very quickly, the simple models Logistic Regression (1.85s total) and Decision Tree (1.44s total) were ideal for real-time or low-power scenarios. Random Forest and Gradient Boosting (ensemble models) delivered strong accuracy (97.70% and 95.73%, respectively) but required considerably more time for training (72.80s and 48.21s). The MLP neural network (35.02s training, 96.67% accuracy) provided a reasonable deep-learning alternative. XGBoost was particularly efficient, with a training time of 3.84s, highest accuracy (97.79%) and total time of 3.88s. By contrast, the Support Vector Machine was extremely slow (696.47s training, 11.72s prediction, 708.19s total), and unsuitable for large-scale and real-time IDS.

Table 4.14. Training, Prediction, and Total Time for Selected Six Models

Model	**Training Time (s)**	**Prediction Time (s)**	**Total Time (s)**
Logistic Regression	1.85	0.00	1.85
kNN	0.01	6.17	6.18
Decision Tree	1.44	0.00	1.44
Random Forest	72.80	0.45	73.25
Gradient Boosting	48.21	0.02	48.23
MLP	35.02	0.01	35.03

4.9. Computational Cost and Latency

Table 4.15 summaries the training time, prediction latency, and total execution duration for all eight methods, now including the SVM and XGBoost classifiers. A visual representation of these timing metrics is provided in Figure 4.8.simple models such as Logistic Regression (1.85s training) and Decision Tree (1.44s total) train very quickly and have low prediction times, making them suitable for low-resource or real-time environments; k-Nearest Neighbors (kNN) trains very fast (0.01s) but has a high prediction time (6.17s). Ensemble learners (Random Forest 72.80s, Gradient Boosting 48.21s) have high accuracy (97.70% and 95.73%, respectively) but are slow to train; the Multi-Layer Perceptron (35.02s) strikes a compromise. XGBoost is the most efficient high-accuracy model, with short training times (3.84s) and the highest accuracy (97.79%) with a total time of 3.88 s. By contrast, the Support Vector Machine model is computationally demanding (696.47s training time, 11.72s prediction time) and unsuitable for real-time IDS. In summary, selecting an IDS model involves weighing the trade-offs between detection accuracy and training and prediction times, depending on the specific use case.

Table 4.15. Training, inference, and total execution time for the selected six classifiers.

Classifier	**Training Duration (s)**	**Prediction Latency (s)**	**Total Elapsed Time (s)**
Logistic Regression	1.85	0.00	1.85
k-NN	0.01	6.17	6.18
Decision Tree	1.44	0.00	1.44
Random Forest	72.80	0.45	73.25
Gradient Boosting	48.21	0.02	48.23
MLP	35.02	0.01	35.03

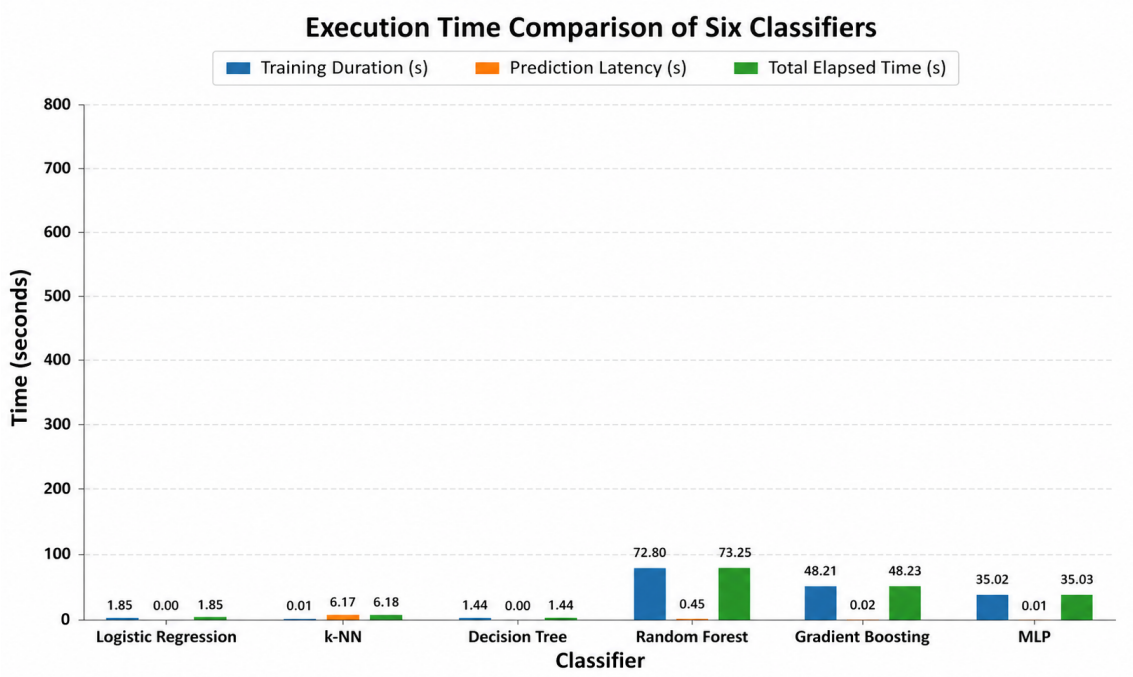

Fig. 4.8. Training, Prediction, and Total Time of Models in a graph

4.10. Configuration of the Machine Learning Models

Machine learning algorithms were implemented and evaluated largely using the scikit-learn library (except for the neural network, which was evaluated with the Keras API). Our baseline results were obtained by running Logistic Regression, k-Nearest Neighbours (with 9 neighbours), Decision Tree and Gradient Boosting using their default hyperparameters. The Random Forest ensemble was configured with 600 trees and parallelisation of jobs, which took advantage of multi-core processing to improve the model's accuracy and to speed up the training process.The Multi-Layer Perceptron (MLP) network was constructed with two hidden layers comprise of 20 neurons each and used an Rectified Linear Unit (ReLU) activation function. Mini-batch training of size 1000 was selected to speed up training through stochastic gradient descent. The Support Vector Machine (SVM) network was tested using a Radial Basis Function (RBF) kernel, regularisation constant $C = 1.0$, and gamma set to `'scale'` so that it is scaled to the features. XGBoost was run with its default settings, leveraging its optimised gradient boosting algorithm. This standardised experimental setup ensured a level playing field among models with regard to parameterization, performance, and run time.

4.11. Machine Learning Bolsters Anomaly Detection

Machine learning approaches can significantly improve the accuracy of network anomaly detection by using feature selection and normalisation to enhance the quality of the training data. With the availability of labelled data, supervised classifiers (Random Forest,

XGBoost, SVM) can achieve high accuracy in classifying network traffic. However, unsupervised approaches such as Isolation Forest and autoencoders detect abnormalities without relying on labelled data, making them suitable for detecting zero-day attacks.Deep learning models, such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), are particularly effective in capturing spatial and temporal features of network traffic. Additionally, addressing class imbalances with resampling techniques (e.g., SMOTE) increases the accuracy of models, particularly for minority classes. Integrating these complementary techniques enhances the adaptability and efficacy of intrusion detection systems.

5. CONCLUSIONS AND FUTURE DIRECTIONS

5.1. Conclusions

This study undertook a comprehensive analysis of an intrusion detection system based on the UNSW-NB15 dataset. The performance of eight different machine learning models was compared, with respect to training time, inference time, accuracy, precision, recall, and F1-score. XGBoost achieved the best classification accuracy (97.79%) and was very closely followed by Random Forest (97.70%). While Random Forest was highly accurate, it came at the cost of the highest training time (72.80s), while XGBoost was more efficient, with a training time of 3.84s and comparable accuracy. Both Decision Tree (96.68%) and the Multi-Layer Perceptron (96.67%) offered high accuracy with low computational complexity; as such they proved to be well-suited for low-power applications. K-Nearest Neighbors (94.84%) and Gradient Boosting (95.73%) also demonstrated good results, yet suffered from either high inference time or training cost, respectively. The Support Vector Machine achieved respectable accuracy (94.91%), but required an unreasonable computational cost (696.47s for training and 11.72s for prediction), and so was excluded from consideration for real-time applications.These results demonstrate the trade-off between the accuracy and computational costs of detection in selecting an IDS model. Although Logistic Regression achieved the lowest accuracy (92.56%), it had the shortest training time (1.85s) and zero inference latency, and thus can serve as an efficient baseline or can be used in scenarios with limited computational resources. Ensemble models such as Random Forest and Gradient Boosting presented greater computational costs, while Decision Tree and Logistic Regression offered a good trade-off.KNN had the highest inference delay (6.17s), which hampers its deployment in real-time applications. XGBoost was the most well-balanced model, with the highest accuracy and a combined training and prediction time of 3.88s. In conclusion, the research highlights the need to consider detection accuracy, computational time, and real-time requirements when choosing the right machine learning model for cybersecurity systems.

5.2. Future Research

Even though the models evaluated in this study showed encouraging intrusion detection capabilities, there are still chances for further advancements. First, to successfully capture intricate attack patterns, more advanced deep learning models (Convolutional Neural Networks, Recurrent Neural Networks with Long Short-Term Memory) may be used. Second, to speed up the modelling process without sacrificing the essential discriminative features, sophisticated feature engineering techniques like Principal Component Analysis and autoencoder-based feature compression might be investigated. Thirdly, hybrid mod-

els that integrate deep neural networks and conventional machine learning may be created to boost performance and increase interpretability. To verify these models' feasibility, it is essential to test them in actual high-throughput network situations. Future research should look at adversarial resistance to make sure detection algorithms are resilient to adversaries' evasion tactics. To further improve latency in resource-constrained contexts, methods including edge-level deployment, quantisation, and model pruning can be explored. Lastly, as they may be able to identify zero-day attacks with little or no labelled data, semi-supervised and unsupervised approaches should be investigated. When combined, these steps will aid in the development of more resilient, scalable, and effective intrusion detection systems to counter the always changing range of cybersecurity risks.

BIBLIOGRAPHY

Abdulhammed, R., Musafer, H., Alessa, A., Faezipour, M., & Abuzneid, A. (2019). Features dimensionality reduction approaches for machine learning based network intrusion detection. *Electronics*, *8*(3), 322.

Abiodun, O. I., Abiodun, E. O., Alawida, M., Alkhawaldeh, R. S., & Arshad, H. (2021). A review on the security of the internet of things: Challenges and solutions. *Wireless Personal Communications*, *119*, 2603–2637.

Al-Naymat, G., Al-Kasassbeh, M., Abu-Samhadanh, N., & Sakr, S. (2016). Classification of voip and non-voip traffic using machine learning approaches. *Journal of Theoretical & Applied Information Technology*.

Belgacem, R., Malek, I. T., Trabelsi, H., & Jabri, I. (2018). A supervised machine learning algorithm skvms used for both classification and screening of glaucoma disease. *New Front. Ophthalmol*, *4*(4), 1–27.

Berthier, R., & Sanders, W. H. (2011). Specification-based intrusion detection for advanced metering infrastructures. *2011 IEEE 17th Pacific rim international symposium on dependable computing*, 184–193.

Chaabouni, N., Mosbah, M., Zemmari, A., Sauvignac, C., & Faruki, P. (2019). Network intrusion detection for iot security based on learning techniques. *IEEE Communications Surveys & Tutorials*, *21*(3), 2671–2701.

Cunningham, P., & Delany, S. J. (2021). K-nearest neighbour classifiers-a tutorial. *ACM computing surveys (CSUR)*, *54*(6), 1–25.

Erhan, L., Ndubuaku, M., Di Mauro, M., Song, W., Chen, M., Fortino, G., Bagdasar, O., & Liotta, A. (2021). Smart anomaly detection in sensor systems: A multi-perspective review. *Information Fusion*, *67*, 64–79.

Falowo, O. I., Ozer, M., Li, C., & Abdo, J. B. (2024). Evolving malware & ddos attacks: Decadal longitudinal study. *IEEE Access*.

Ferrag, M. A., Friha, O., Hamouda, D., Maglaras, L., & Janicke, H. (2022). Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning. *IEEe Access*, *10*, 40281–40306.

Gharaee, H., & Hosseinvand, H. (2016). A new feature selection ids based on genetic algorithm and svm. *2016 8th International Symposium on Telecommunications (IST)*, 139–144.

Hafeez, M. A., Rashid, M., Tariq, H., Abideen, Z. U., Alotaibi, S. S., & Sinky, M. H. (2021). Performance improvement of decision tree: A robust classifier using tabu search algorithm. *Applied Sciences*, *11*(15), 6728.

Hotzy, F., Theodoridou, A., Hoff, P., Schneeberger, A. R., Seifritz, E., Olbrich, S., & Jäger, M. (2018). Machine learning: An approach in identifying risk factors for coercion compared to binary logistic regression. *Frontiers in psychiatry*, *9*, 258.

Jaiswal, J. K., & Samikannu, R. (2017). Application of random forest algorithm on feature subset selection and classification and regression. *2017 world congress on computing and communication technologies (WCCCT)*, 65–68.

Jiang, K., Wang, W., Wang, A., & Wu, H. (2020). Network intrusion detection combined hybrid sampling with deep hierarchical network. *IEEE access*, *8*, 32464–32476.

Jiang, Z., Crookes, D., Green, B. D., Zhao, Y., Ma, H., Li, L., Zhang, S., Tao, D., & Zhou, H. (2018). Context-aware mouse behavior recognition using hidden markov models. *IEEE Transactions on Image Processing*, *28*(3), 1133–1148.

Kessler, T., Dorian, G., & Mack, J. H. (2017). Application of a rectified linear unit (relu) based artificial neural network to cetane number predictions. *Internal combustion engine division fall technical conference*, *58318*, V001T02A006.

Khammassi, C., & Krichen, S. (2017). A ga-lr wrapper approach for feature selection in network intrusion detection. *computers & security*, *70*, 255–277.

Khan, M. Y., Qayoom, A., Nizami, M. S., Siddiqui, M. S., Wasi, S., & Raazi, S. M. K.-R. (2021). Automated prediction of good dictionary examples (gdex): A comprehensive experiment with distant supervision, machine learning, and word embedding-based deep learning techniques. *Complexity*, *2021*(1), 2553199.

Manjunatha, B., Gogoi, P., & Akkalappa, M. (2019). Data mining based framework for effective intrusion detection using hybrid feature selection approach. *International Journal of Computer Network and Information Security*, *11*(8), 1.

Moukhafi, M., Yassini, K. E., Bri, S., & Oufaska, K. (2019). Artificial neural network optimized by genetic algorithm for intrusion detection system. *Advanced Intelligent Systems for Sustainable Development (AI2SD'2018) Volume 5: Advanced Intelligent Systems for Computing Sciences*, 393–404.

Moustafa, N. (2021). A new distributed architecture for evaluating ai-based security systems at the edge: Network ton_iot datasets. *Sustainable Cities and Society*, *72*, 102994.

Moustafa, N., & Slay, J. (2015). Unsw-nb15: A comprehensive data set for network intrusion detection systems (unsw-nb15 network data set). *2015 military communications and information systems conference (MilCIS)*, 1–6.

Moustafa, N., & Slay, J. (2016). The evaluation of network anomaly detection systems: Statistical analysis of the unsw-nb15 data set and the comparison with the kdd99 data set. *Information Security Journal: A Global Perspective*, *25*(1-3), 18–31.

Mphatheni, M. R., & Maluleke, W. (2022). Cybersecurity as a response to combating cybercrime: Demystifying the prevailing threats and offering recommendations to the african regions. *International Journal of Research in Business and Social Science (2147-4478)*, *11*(4), 384–396.

Natekin, A., & Knoll, A. (2013). Gradient boosting machines, a tutorial. *Frontiers in neurorobotics*, *7*, 21.

Nobel, S. N., Sultana, S., Singha, S. P., Chaki, S., Mahi, M. J. N., Jan, T., Barros, A., & Whaiduzzaman, M. (2024). Unmasking banking fraud: Unleashing the power of

machine learning and explainable ai (xai) on imbalanced data. *Information*, *15*(6), 298.

Ouafiq, E. M., Saadane, R., & Chehri, A. (2022). Data management and integration of low power consumption embedded devices iot for transforming smart agriculture into actionable knowledge. *Agriculture*, *12*(3), 329.

Parampottupadam, S., & Moldovann, A.-N. (2018). Cloud-based real-time network intrusion detection using deep learning. *2018 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, 1–8.

Patel, A., Qassim, Q., & Wills, C. (2010). A survey of intrusion detection and prevention systems. *Information Management & Computer Security*, *18*(4), 277–290.

Portugal, I., Alencar, P., & Cowan, D. (2018). The use of machine learning algorithms in recommender systems: A systematic review. *Expert Systems with Applications*, *97*, 205–227.

Redhu, A., Choudhary, P., Srinivasan, K., & Das, T. K. (2024). Deep learning-powered malware detection in cyberspace: A contemporary review. *Frontiers in Physics*, *12*, 1349463.

Ren, J., Guo, J., Qian, W., Yuan, H., Hao, X., & Jingjing, H. (2019). Building an effective intrusion detection system by using hybrid data optimization based on machine learning algorithms. *Security and communication networks*, *2019*(1), 7130868.

Reveron, D. S. (2012). *Cyberspace and national security: Threats, opportunities, and power in a virtual world*. Georgetown University Press.

Riedmiller, M., & Lernen, A. (2014). Multi layer perceptron. *Machine learning lab special lecture, University of Freiburg*, *24*.

Salman, T., Bhamare, D., Erbad, A., Jain, R., & Samaka, M. (2017). Machine learning for anomaly detection and categorization in multi-cloud environments. *2017 IEEE 4th international conference on cyber security and cloud computing (CSCloud)*, 97–103.

Sarhan, M., Layeghy, S., & Portmann, M. (2021). Feature analysis for machine learning-based iot intrusion detection. *arXiv preprint arXiv:2108.12732*.

Sethi, K., Kumar, R., Prajapati, N., & Bera, P. (2020). Deep reinforcement learning based intrusion detection system for cloud infrastructure. *2020 International Conference on COMmunication Systems & NETworkS (COMSNETS)*, 1–6.

Sharma, J., Giri, C., Granmo, O.-C., & Goodwin, M. (2019). Multi-layer intrusion detection system with extratrees feature selection, extreme learning machine ensemble, and softmax aggregation. *EURASIP Journal on Information Security*, *2019*(1), 1–16.

Tausendschön, J., Stoeckl, G., & Radl, S. (2023). Machine learning for heat radiation modeling of bi-and polydisperse particle systems including walls. *Particuology*, *74*, 119–140.

Wu, P., Moustafa, N., Yang, S., & Guo, H. (2020). Densely connected residual network for attack recognition. *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 233–242.

Yi, D., Ahn, J., & Ji, S. (2020). An effective optimization method for machine learning based on adam. *Applied Sciences*, *10*(3), 1073.

Zaidi, A., & Al Luhayb, A. S. M. (2023). Two statistical approaches to justify the use of the logistic function in binary logistic regression. *Mathematical Problems in Engineering*, *2023*(1), 5525675.

Zhang, S., Li, X., Zong, M., Zhu, X., & Cheng, D. (2017). Learning k for knn classification. *ACM Transactions on Intelligent Systems and Technology (TIST)*, *8*(3), 1–19.

Zhang, W., et al. (2017). Machine learning approaches to predicting company bankruptcy. *Journal of Financial Risk Management*, *6*(04), 364.